

Netra

Offline facial recognition & liveness for zero-network field authentication

NHAI Hackathon 7.0

Fully offline · React Native (Android + iOS) · ≈ 10 MB models · CPU-only

The problem

Field personnel must be authenticated in **remote, zero-network** locations on **standard mid-range phones**, and it must drop into the existing **Datalake 3.0** React Native app.

Requirements:

- **Offline** face recognition + **liveness** (anti-spoofing)
- Lightweight model (**target ≤ 20 MB**), **< 1 s** recognition
- Android 8.0+ / iOS 12+, 3 GB RAM, **no GPU**
- **Open-source only**; sync + **purge** to server when network returns

Solution at a glance

```
Camera --> MediaPipe FaceMesh --> Liveness gate --> 112x112 crop
      --> MobileFaceNet (TFLite, FP16) --> 512-d embedding
      --> cosine match vs encrypted on-device template --> ✓ / ✗
                    |
                    v
      auth event --> encrypted offline queue --> (on reconnect) sync --> purge
```

- **100% on-device** authentication — no network needed to enroll or verify
- Edge AI: **MobileFaceNet + ArcFace**, FP16, **6.5 MB**
- Liveness: **randomized blink / smile / head-turn**, all offline
- **Sync & purge** audit lifecycle to an AWS-deployable backend

Innovation — edge model & compression

Asset	Size
MobileFaceNet/ArcFace (WebFace600K), FP16	6.5 MB
MediaPipe Face Landmarker	3.6 MB
Total footprint	≈ 10.1 MB — ~50% of the 20 MB budget

- **ArcFace** angular-margin embeddings → robust across **diverse Indian demographics** and outdoor lighting (model family reports >99% on LFW-style).
- **FP16 quantization** halves size with negligible accuracy loss.
- **512-d, L2-normalized** → stable cosine matching; multi-frame **centroid** enrollment (front/left/right) for pose/lighting robustness.

Offline liveness (anti-spoofing)

One challenge is drawn at random per login — blink / smile / turn-left / turn-right — and **only that one** is accepted, so a replayed clip of a different gesture fails. Each is a **scale-invariant** change in FaceMesh geometry:

- **Blink** = Eye-Aspect-Ratio dips closed (≤ 0.19) then opens (≥ 0.27) across frames. A photo/screen holds constant EAR → **fails**.
- **Smile** = mouth-width / inter-ocular ratio goes neutral (≤ 0.50) → widened (≥ 0.58). A fixed smile in a photo holds a constant ratio → **fails**.
- **Head-turn** = signed nose offset / eye-distance ≥ 0.07 ; direction (left vs right) is preserved and reported to the backend.

The match is gated — login refuses to run recognition until liveness passes, defeating the photo/replay attack. Logic is **pure & unit-tested**.

Feasibility — speed < 1 s, easy integration

Measured on-device (Samsung SM-F956B, Android 16, **CPU-only**), 20 runs, median:

detect	crop+norm	infer	match	core (recog+liveness)
177 ms	292 ms	20 ms	~0 ms	501 ms ✓ (< 1 s)

- **MobileFaceNet inference is ~20 ms** — the AI core is trivially fast; the rest is FaceMesh + preprocessing. Camera capture (~728 ms) is hardware-bound, excluded.
- **No GPU** (`default` TFLite delegate). Reproduce live: Home → Benchmark.
- **Drop-in SDK:** `src/faceAuth/sdk` exposes `embed/match/liveness/enroll/sync` without our screens. See `docs/INTEGRATION.md` for the Datalake-3.0 guide.

Scalability & Sustainability — Sync & Purge

Offline events are durably queued (encrypted), then on reconnect:

1. `POST /sync/events` → server confirms `acceptedEventIds`
2. `POST /sync/purge-ack` → server marks **PURGED** (kept for audit)
3. Device **deletes the local row — ACK-before-purge**
 - Triggers: **NetInfo reconnect**, retry interval, app-foreground
 - Visible live on the **Sync & Purge** screen ("Synced & purged" counter)
 - Backend: Go + MongoDB, **AWS-deployable** (ECS/EC2 + DocumentDB)

Architecture

Layer	Tech
Mobile app	React Native 0.80 (TS), VisionCamera
Recognition	MobileFaceNet/ArcFace, <code>react-native-fast-tflite</code>
Geometry/liveness	MediaPipe Face Landmarker (478 pts)
Secure storage	EncryptedSharedPreferences / iOS Keychain
Backend	Go + Gin + MongoDB, Ed25519-signed profiles, JWT
Admin panel	React + Vite

Cross-platform: native modules for **both Android (Kotlin) and iOS (Swift)**.

Open-source & compliance

- **MobileFaceNet/ArcFace** — InsightFace, **MIT**
- **MediaPipe Face Landmarker** — Google, **Apache-2.0**
- TFLite, VisionCamera, React Native, NetInfo — **MIT / Apache-2.0**
- **No additional licenses required.** Full attribution in `THIRD_PARTY_LICENSES.md`.

Meets: Android 8.0+/iOS 12+ · 3 GB RAM · no GPU · ≤ 20 MB · offline.

Roadmap

- Continuous-frame liveness processing (beyond capture-loop sampling)
- On-device FAR/FRR study across demographics & lighting
- Native iOS TFLite delegate tuning (Core ML) for extra headroom
- Hard-delete purged events server-side after a retention window

Netra — Thank you

Offline. Lightweight. Secure. Cross-platform.

Source + docs: `README.md` · `docs/TECHNICAL_DOCUMENTATION.md` ·
`docs/INTEGRATION.md`